LESSON 02

Broken Authentication

JWT Forgery & Signature Bypass in AWS Lambda

OWASP API2

Severity: Critical

Status: Fixed ✓

VULNERABILITY OVERVIEW

What Is the Problem?

Affected Component

DVSA-ORDER-MANAGER
AWS Lambda function

Root Cause

JWT payload decoded but
signature never verified

Impact

Full account takeover —
any user's orders stolen

JWT Structure:

HEADER	PAYLOAD	SIGNATURE ✗ (never

The backend only decoded the PAYLOAD — it never verified the SIGNATURE

ROOT CAUSE

The Vulnerable Code

order-manager.js (VULNERABLE)

```
// ❌ No verification — just split and decode
var token_sections = auth_header.split('.');
var auth_data = jose.util.base64url.decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;  // ← TRUSTED WITHOUT VERIFICATION
```

Attacker can PUT ANY username here — backend trusts it blindly

Signature section is ignored entirely — no cryptographic check

EXPLOITATION

Attack Steps

1

Register & Login as User B

Attacker creates their own account and places an order. Captures JWT from DevTools Network tab.

2

Decode Token → Extract Victim UUID

Base64-decode the JWT payload to reveal the username/sub fields. Run same decode on User C's token to get victim UUID.

3

Forge a New JWT

Replace username and sub with User C's UUID. Keep original header and signature — backend never checks them.

4

Send Forged Token → Steal Orders

Call the Orders API with $FAKE_AS_C. Backend returns User C's full order list — exploit confirmed.

EVIDENCE & PROOF

Exploit Confirmed

✓ Normal Behavior

```
Token B queries orders →
returns ONLY User B's orders

Order: Rampage (Nintendo)
Total: $40
Order ID: 90ae1117...
```

⭐ Exploit Proof

```
Forged token (User C UUID) →
returns User C's orders!

Order: Super Mario 2 (Nintendo)
Total: $45
Order ID: 15fd86f0...

← User B stole User C's data
```

REMEDIATION

The Fix Applied

1. **Fetch Cognito JWKS**
   `https://cognito-idp.{region}.amazonaws.com/{poolId}/.well-known/jwks.json`

2. **Cache public keys (6hr TTL)**
   `getCognitoKeystore() — avoids redundant network calls per invocation`

3. **Verify JWT signature**
   `jose.JWS.createVerify(keystore).verify(jwt) — cryptographic check`

4. **Validate claims (iss, exp)**
   `Issuer must match Cognito pool URL. Token must not be expired.`

File changed: DVSA-ORDER-MANAGER/order-manager.js

VERIFICATION

After Fix — Results

Forged Token → REJECTED ✓

```
curl -H "authorization: $FAKE_AS_C" --data-raw '{"action":"orders"}'
```

```
{
  "status": "err",
  "msg": "invalid token"
}
```

Legitimate Token → WORKS ✓

```
curl -H "authorization: $TOKEN_B" --data-raw '{"action":"orders"}'
```

```
{
  "status": "ok",
  "orders": [ { "order-id":
"90ae1117...",
    "total": 40 } ]
}
```

Tables A & B

Table A — Operation Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Auth	Only verified JWTs may determine identity	DevTools, JWT decode, API responses, source code	TOKEN_B returns only User B's orders	FAKE_AS_C returns User C's orders

Table B — Deviation and Fix Analysis

Vulnerability	Why a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Broken Auth	Attacker-controlled claims used to authorize access without Cognito issuing token	Intentional misuse / security-relevant abuse	verifyCognitoJwt() added to order-manager.js; validates sig, iss, exp via JWKS	Forged token rejected. Legit token still works.

Key Takeaway

Never trust JWT payload claims without cryptographic signature verification. In AWS, always validate against the Cognito JWKS endpoint. Serverless functions inherit all traditional auth vulnerabilities — a single missing check enables full account takeover for every user in the system.

LESSON 02 COMPLETE

Summary

❌ Vulnerability: JWT payload trusted without signature verification in DVSA-ORDER-MANAGER

⚔️ Exploit: Forged token with victim UUID returned full order history (cross-account data theft)

🔧 Fix: verifyCognitoJwt() — JWKS fetch, signature check, issuer + expiry validation

✅ Verified: Forged token rejected; legitimate token still works correctly